SNES Assembly Adventure 11: Initialization and Project Template

19 May 2022

Welcome back, Adventurer! Last time, we learned how to use CMake and cc65 to automate the build process.

This time we'll refactor the modular example from the previous article to become a more general project template we can use in the future. This project template will

- have a handful of helper subroutines for loading and clearing VRAM, CGRAM, and OAM, handling the joypad;
- initialize the SNES to be in a defined state; i.e., clear all memory and set registers to predefined values;
- have a simple main game loop and a v-blank loop that does nothing.

The final project template can be found on GitHub.



We'll only make small changes to the overall structure. Let's go.

What We Got So Far

Currently, the directory structure looks like this:

```
snes_cradle/
    assets/
        Assets.inc
        Assets.s
        CMakeLists.txt
        SpriteColors.pal
        Sprites.vra
    cmake/
        CMakeCA65816Compiler.cmake.in
        CMakeCA65816Information.cmake
        CMakeDetermineCA65816Compiler.cmake
        CMakeTestCA65816Compiler.cmake
    src/
```

```
        common/
            CMakeLists.txt
            GameConstants.inc
            MemoryMapWRAM.inc
            Registers.inc
            Vector.s
        Game/
            CMakeLists.txt
            Init.inc
            Init.s
        PPU/
            CMakeLists.txt
            PPU.inc
            PPU.s
        CMakeLists.txt
        JoypadSprite.s
    CMakeLists.txt
    MemoryMap.cfg
```

The most changes we'll make are to `src/Game/Init.s` and `src/JoypadSprite.s`. Then we'll add a set of new files to make the code to read the joypads more reusable.

Refactor `JoypadSprites.s`

Here's what we got currently have in `src/JoypadSprites.s`:

```
1  ; ------------------------------------------------------------------------------
2  ;   File: JoypadSprite.s
3  ;   Description: Displays a sprite that bounces off the edges
4  ; ------------------------------------------------------------------------------
5
6  ;----- Export ----------------------------------------------------------------
7  .export     ResetHandler        ; export the entry point of the demo
8  .export     NMIHandler
9  ;------------------------------------------------------------------------------
10
11 ;----- Assembler Directives --------------------------------------------------
12 .p816                          ; tell the assembler this is 65816 code
13 ;------------------------------------------------------------------------------
```

```
14
15  ;----- Includes -----------------------------------------------------------
16  .include "Assets.inc"
17  .include "GameConstants.inc"
18  .include "Init.inc"
19  .include "MemoryMapWRAM.inc"
20  .include "PPU.inc"
21  .include "Registers.inc"
22  ;--------------------------------------------------------------------------
23
24  .segment "CODE"
25  ;--------------------------------------------------------------------------
26  ;   This is the entry point of the demo
27  ;--------------------------------------------------------------------------
28  .proc   ResetHandler
29          sei                     ; disable interrupts
30          clc                     ; clear the carry flag
31          xce                     ; switch the 65816 to native (16-bit mode)
32          rep #$10                ; set X and Y to 16-bit
33          sep #$20                ; set A to 8-bit
34          lda #$8f                ; force v-blanking
35          sta INIDISP
36          stz NMITIMEN            ; disable NMI
37          ; set the stack pointer to $1fff
38          ldx #$1fff              ; load X with $1fff
39          txs                     ; copy X to stack pointer
40          jsr InitDemo            ; initialize the demo
41          jmp GameLoop            ; enter main game loop
42  .endproc
43  ;--------------------------------------------------------------------------
44
45  ;--------------------------------------------------------------------------
46  ;   Executed during V-Blank
47  ;--------------------------------------------------------------------------
48  .proc   GameLoop
49          wai                                 ; wait for NMI / V-Blank
50
51          ; read joypad 1
52          ; check whether joypad is ready
53  WaitForJoypad:
54          lda HVBJOY                          ; get joypad status
55          and #$01                            ; check whether joypad done readin
56          beq WaitForJoypad                   ; ...if not, wait a bit more
57          ; first, check for newly pressed buttons since last frame
```

```
58          rep #$20                        ; set A to 16-bit
59          lda STDCNTRL1L                  ; get new input from this frame
60          ldy JOYPAD1                     ; get input from last frame
61          sta JOYPAD1                     ; store new input from this frame
62          tya                             ; check for newly pressed buttons.
63          eor JOYPAD1                     ; filter buttons that were not pre
64          and JOYPAD1                     ; filter held buttons from last fr
65          sta JOYTRIGGER1                 ; ...and store them
66          ; second, check for buttons held from last frame
67          tya                             ; get input from last frame
68          and JOYPAD1                     ; filter held buttons from last fr
69          sta JOYHELD1                    ; ...store them
70
71          ; check the dpad, if any of the directional buttons where pressed or h
72          ; move the sprites accordingly
73  CheckUpButton:
74          lda #$0000                      ; set A to zero
75          ora JOYTRIGGER1                 ; check whether the up button was
76          ora JOYHELD1                    ; ...or held from last frame
77          and #UP_BUTTON
78          beq CheckUpButtonDone           ; if neither has occured, move on
79          ; else, move sprites up
80          ldy #$0000                      ; Y is the loop counter
81          ldx #$0001                      ; set offset to 1, to manipulate s
82          sep #$20                        ; set A to 8-bit
83  MoveSpritesUp:
84          lda OAMMIRROR, X
85          sec
86          sbc VER_SPEED
87          bcc CorrectVerticalPositionDown ; if vertical position is below ze
88          sta OAMMIRROR, X
89          inx                             ; increment X by 4
90          inx
91          inx
92          inx
93          iny
94          cpy #$0004                      ; unless Y = 4, continue loop
95          bne MoveSpritesUp
96  CheckUpButtonDone:
97          rep #$20                        ; set A to 16-bit
98
99  CheckDownButton:
100         lda #$0000                      ; set A to zero
101         ora JOYTRIGGER1                 ; check whether the down button wa
```

```
102         ora JOYHELD1                        ; ...or held from last frame
103         and #DOWN_BUTTON
104         beq CheckDownButtonDone             ; if neither has occured, move on
105         ; else, move sprites down
106         ldy #$0000                          ; Y is the loop counter
107         ldx #$0001                          ; set offset to 1, to manipulate s
108         sep #$20                            ; set A to 8-bit
109         ; check if sprites move below buttom boundry
110         lda OAMMIRROR, X
111         clc
112         adc VER_SPEED
113         cmp #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
114         bcs CorrectVerticalPositionUp
115 MoveSpritesDown:
116         lda OAMMIRROR, X
117         clc
118         adc VER_SPEED
119         sta OAMMIRROR, X
120         inx                                 ; increment X by 4
121         inx
122         inx
123         inx
124         iny
125         cpy #$0004                          ; unless Y = 4, continue loop
126         bne MoveSpritesDown
127 CheckDownButtonDone:
128         rep #$20                            ; set A to 16-bit
129         jmp CheckLeftButton                 ; continue input check
130
131 CorrectVerticalPositionDown:
132         sep #$20                            ; set A to 8-bit
133         stz OAMMIRROR + 1                   ; sprite 1, vertical position
134         stz OAMMIRROR + 5                   ; sprite 3, vertical position
135         lda #SPRITE_SIZE
136         sta OAMMIRROR + 9                   ; sprite 2, vertical position
137         sta OAMMIRROR + 13                  ; sprite 4, vertical position
138         rep #$20                            ; set A to 16-bit
139         jmp CheckLeftButton                 ; continue input check
140
141 CorrectVerticalPositionUp:
142         sep #$20                            ; set A to 8-bit
143         lda #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
144         sta OAMMIRROR + 1                   ; sprite 1, vertical position
145         sta OAMMIRROR + 5                   ; sprite 3, vertical position
```

```
146         lda #(SCREEN_BOTTOM - SPRITE_SIZE)
147         sta OAMMIRROR + 9                   ; sprite 2, vertical position
148         sta OAMMIRROR + 13                  ; sprite 4, vertical position
149         rep #$20                            ; set A to 16-bit
150
151 CheckLeftButton:
152         lda #$0000                          ; set A to zero
153         ora JOYTRIGGER1                     ; check whether the up button was
154         ora JOYHELD1                        ; ...or held from last frame
155         and #LEFT_BUTTON
156         beq CheckLeftButtonDone             ; if neither has occured, move on
157         ; else, move sprites up
158         ldy #$0000                          ; Y is the loop counter
159         ldx #$0000                          ; set offset to 0, to manipulate s
160         sep #$20                            ; set A to 8-bit
161 MoveSpritesLeft:
162         lda OAMMIRROR, X
163         sec
164         sbc HOR_SPEED
165         bcc CorrectHorizontalPositionRight
166         sta OAMMIRROR, X
167         inx                                 ; increment X by 4
168         inx
169         inx
170         inx
171         iny
172         cpy #$0004                          ; unless Y = 4, continue loop
173         bne MoveSpritesLeft
174 CheckLeftButtonDone:
175         rep #$20                            ; set A to 16-bit
176
177 CheckRightButton:
178         lda #$0000                          ; set A to zero
179         ora JOYTRIGGER1                     ; check whether the down button wa
180         ora JOYHELD1                        ; ...or held from last frame
181         and #RIGHT_BUTTON
182         beq CheckRightButtonDone            ; if neither has occured, move on
183         ; else, move sprites down
184         ldy #$0000                          ; Y is the loop counter
185         ldx #$0000                          ; set offset to 0, to manipulate s
186         sep #$20                            ; set A to 8-bit
187         ; check whether sprites move beyond right boundry
188         lda OAMMIRROR, X
189         clc
```

```
190         adc HOR_SPEED
191         cmp #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
192         bcs CorrectHorizontalPositionLeft
193 MoveSpritesRight:
194         lda OAMMIRROR, X
195         clc
196         adc HOR_SPEED
197         sta OAMMIRROR, X
198         inx                             ; increment X by 4
199         inx
200         inx
201         inx
202         iny
203         cpy #$0004                      ; unless Y = 4, continue loop
204         bne MoveSpritesRight
205 CheckRightButtonDone:
206         rep #$20                        ; set A to 16-bit
207         jmp InputDone
208
209 CorrectHorizontalPositionRight:
210         sep #$20                        ; set A to 8-bit
211         stz OAMMIRROR + 0               ; sprite 1, horizontal position
212         stz OAMMIRROR + 8               ; sprite 2, horizontal position
213         lda #SPRITE_SIZE
214         sta OAMMIRROR + 4               ; sprite 3, horizontal position
215         sta OAMMIRROR + 12              ; sprite 4, horizontal position
216         jmp InputDone
217
218 CorrectHorizontalPositionLeft:
219         sep #$20                        ; set A to 8-bit
220         lda #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
221         sta OAMMIRROR + 0               ; sprite 1, horizontal position
222         sta OAMMIRROR + 8               ; sprite 2, horizontal position
223         lda #(SCREEN_RIGHT - SPRITE_SIZE)
224         sta OAMMIRROR + 4               ; sprite 3, horizontal position
225         sta OAMMIRROR + 12              ; sprite 4, horizontal position
226
227 InputDone:
228         sep #$20                        ; set A back to 8-bit
229
230         jmp GameLoop
231 .endproc
232 ;-------------------------------------------------------------------------------
233
```

```
234  ;-------------------------------------------------------------------------
235  ;   Will be called during V-Blank every frame
236  ;-------------------------------------------------------------------------
237  .proc   NMIHandler
238          lda RDNMI               ; read NMI status, acknowledge NMI
239
240          tsx                     ; save old stack pointer
241          pea OAMMIRROR           ; push mirror address to stack
242          jsr UpdateOAMRAM        ; update OAMRAM
243          txs                     ; restore old stack pointer
244
245          rti
246  .endproc
247  ;-------------------------------------------------------------------------
248
249  ;-------------------------------------------------------------------------
250  ;   Is not used in this program
251  ;-------------------------------------------------------------------------
252  .proc   IRQHandler
253          ; code
254          rti
255  .endproc
256  ;-------------------------------------------------------------------------
```

JoypadSprite.s hosted with ❤ by **GitHub** **view raw**

The code we wrote to read the joypad in part 8 is embedded in the main game loop `GameLoop` (line 48). While that's useful for a simple demo, usually, we'd want our input handling to be its own subroutine. So we'll extract this section of code and make it a bit more general purpose - that is, it will also be able to handle the second player/joypad.

Since input handling is common functionality, create two new files `src/Common/Joypad.{inc,s}` with the following content:

```
1   .ifndef JOYPAD_INC
2   JOYPAD_INC = 1
3   ; -------------------------------------------------------------------------
4   ;   File: Joypad.inc
5   ;   Description: Subroutines to read the current state of the joypads
6   ; -------------------------------------------------------------------------
7
8   ;----- Import ------------------------------------------------------------
9   .import     ReadJoypad1
10  .import     ReadJoypad2
```

```
11  ;------------------------------------------------------------------------------
12
```

Joypad.inc hosted with ❤ by **GitHub** **view raw**

```
1   ; ------------------------------------------------------------------------------
2   ;   File: Joypad.s
3   ;   Description: Subroutines to read the current state of the joypads
4   ; ------------------------------------------------------------------------------
5
6   ;----- Export ----------------------------------------------------------------
7   .export     ReadJoypad1
8   .export     ReadJoypad2
9   ;------------------------------------------------------------------------------
10
11  ;----- Assembler Directives --------------------------------------------------
12  .p816                           ; tell the assembler this is 65816 code
13  .a8
14  .i16
15  ;------------------------------------------------------------------------------
16
17  ;----- Includes --------------------------------------------------------------
18  .include "MemoryMapWRAM.inc"
19  .include "Registers.inc"
20  ;------------------------------------------------------------------------------
21
22  .segment "CODE"
23  ;------------------------------------------------------------------------------
24  ;   Read and update the state of joypad 1
25  ;   Parameters: Raw: .addr, Trigger: .addr, Held: .addr
26  ;------------------------------------------------------------------------------
27  .proc   ReadJoypad1
28          phx                                 ; save old stack pointer
29          ; create frame pointer
30          phd                                 ; push Direct Register to stack
31          tsc                                 ; transfer Stack to... (via Accumu
32          tcd                                 ; ...Direct Register.
33          ; use constants to access arguments on stack with Direct Addressing
34          Raw         = $07                   ; address to store raw input data
35          Trigger     = $09                   ; address to store triggered butto
36          Held        = $0b                   ; address to store held buttons
37
38          ; check whether joypad is ready
39  WaitForJoypad:
```

```
40          lda HVBJOY                          ; get joypad status
41          and #$01                            ; check whether joypad done readin
42          beq WaitForJoypad                   ; ...if not, wait a bit more
43
44          ; first, check for newly pressed buttons since last frame
45          ; A - current frame
46          ; Y - auxiliar
47          rep #$20                            ; set A to 16-bit
48          lda (Raw)                           ; get last frame's input...
49          tay                                 ; ...and store in Y
50          lda JOY1L                           ; get new input from this frame
51          sta (Raw)                           ; store new input from this frame
52          tya                                 ; check for newly pressed buttons.
53          eor (Raw)                           ; filter buttons that were not pre
54          and (Raw)                           ; filter held buttons from last fr
55          sta (Trigger)                       ; ...and store them
56          ; second, check for buttons held from last frame
57          tya                                 ; get input from last frame
58          and (Raw)                           ; filter held buttons from last fr
59          sta (Held)                          ; ...store them
60
61          ; done
62          sep #$20                            ; set A back to 8-bit
63          pld                                 ; restore D...
64          plx                                 ; ...and X registers
65          rts
66  .endproc
67  ;-------------------------------------------------------------------------
68
69  ;-------------------------------------------------------------------------
70  ;   Read and update the state of joypad 2
71  ;   Parameters: Raw: .addr, Trigger: .addr, Held: .addr
72  ;-------------------------------------------------------------------------
73  .proc   ReadJoypad2
74          phx                                 ; save old stack pointer
75          ; create frame pointer
76          phd                                 ; push Direct Register to stack
77          tsc                                 ; transfer Stack to... (via Accumu
78          tcd                                 ; ...Direct Register.
79          ; use constants to access arguments on stack with Direct Addressing
80          Raw         = $07                   ; address to store raw input data
81          Trigger     = $09                   ; address to store triggered butto
82          Held        = $0b                   ; address to store held buttons
83
```

```
84          ; check whether joypad is ready
85  WaitForJoypad:
86          lda HVBJOY                          ; get joypad status
87          and #$01                            ; check whether joypad done readin
88          beq WaitForJoypad                   ; ...if not, wait a bit more
89
90          ; first, check for newly pressed buttons since last frame
91          ; A - current frame
92          ; Y - auxiliar
93          rep #$20                            ; set A to 16-bit
94          lda (Raw)                           ; get last frame's input...
95          tay                                 ; ...and store in Y
96          lda JOY2L                           ; get new input from this frame
97          sta (Raw)                           ; store new input from this frame
98          tya                                 ; check for newly pressed buttons.
99          eor (Raw)                           ; filter buttons that were not pre
100         and (Raw)                           ; filter held buttons from last fr
101         sta (Trigger)                       ; ...and store them
102         ; second, check for buttons held from last frame
103         tya                                 ; get input from last frame
104         and (Raw)                           ; filter held buttons from last fr
105         sta (Held)                          ; ...store them
106
107         ; done
108         sep #$20                            ; set A back to 8-bit
109         pld                                 ; restore D...
110         plx                                 ; ...and X registers
111
112         rts
113 .endproc
114 ;-------------------------------------------------------------------------
```

Joypad.s hosted with ❤ by **GitHub** **view raw**

The code here is essentially the same as described in part 8 for handling input. We pass the addresses where to store the input data via the stack, as described in part 6. But pay attention to a subtle change we made: we switched the usage of registers A and Y in lines 39 through 59. Since we now pass the addresses of where to store joypad data by stack (in contrast to using "global" address constants), we need to use indirect addressing. Unfortunately, the 65816 instruction set is not orthogonal, i.e., not all addressing modes are supported by all instructions. The `ldy` instruction does not support *direct page indirect* addressing, but `lda` does - so we switch their usage to take advantage of indirect addressing on A.

You'll probably come across this from time to time: you'll have to rethink/refactor some of your logic because not all instructions support all addressing modes. It's especially vexing at times with `lda`, `ldy`, and `ldx` and their storage counterparts. In fact, each

Next, we extract the logic that moves the sprites on the screen. Since this is game logic, we'll create a new file `src/Game/Input.{inc,s}`:

```
.ifndef INPUT_INC
INPUT_INC = 1
; ----------------------------------------------------------------------------
;   File: Input.inc
;   Description: Subroutines to handle input
; ----------------------------------------------------------------------------

;----- Import ----------------------------------------------------------------
.import     HandleInput
;----------------------------------------------------------------------------

.endif
```

Input.inc hosted with ❤ by **GitHub** **view raw**

```
; ----------------------------------------------------------------------------
;   File: Input.s
;   Description: Subroutines to handle input
; ----------------------------------------------------------------------------

;----- Export ----------------------------------------------------------------
.export     HandleInput
;----------------------------------------------------------------------------

;----- Assembler Directives --------------------------------------------------
.p816                           ; tell the assembler this is 65816 code
.a8
.i16
;----------------------------------------------------------------------------

;----- Includes --------------------------------------------------------------
.include "GameConstants.inc"
.include "MemoryMapWRAM.inc"
;----------------------------------------------------------------------------

.segment "CODE"
;----------------------------------------------------------------------------
```

```
23  ;   This subroutines handles all input
24  ;   Parameters: Raw: .addr, Trigger: .addr, Held: .addr
25  ;------------------------------------------------------------------------------
26  .proc   HandleInput
27          phx                             ; save old stack pointer
28          ; create frame pointer
29          phd                             ; push Direct Register to stack
30          tsc                             ; transfer Stack to... (via Accumu
31          tcd                             ; ...Direct Register.
32          ; use constants to access arguments on stack with Direct Addressing
33          Raw         = $07               ; address to store raw input data
34          Trigger     = $09               ; address to store triggered butto
35          Held        = $0b               ; address to store held buttons
36
37          rep #$20                        ; set A to 16-bit
38          ; check the dpad, if any of the directional buttons where pressed or h
39          ; move the sprites accordingly
40  CheckUpButton:
41          lda #$0000                      ; set A to zero
42          ora (Trigger)                   ; check whether the up button was
43          ora (Held)                      ; ...or held from last frame
44          and #UP_BUTTON
45          beq CheckUpButtonDone           ; if neither has occured, move on
46          ; else, move sprites up
47          ldx #$0000                      ; X is the loop counter
48          ldy #$0001                      ; Y is the offset into the OAM mir
49          sep #$20                        ; set A to 8-bit
50  MoveSpritesUp:
51          lda OAMMIRROR, Y
52          sec
53          sbc VER_SPEED
54          bcc CorrectVerticalPositionDown ; if vertical position is below ze
55          sta OAMMIRROR, Y
56          iny                             ; increment Y by 4
57          iny
58          iny
59          iny
60          inx
61          cpx #$0004                      ; unless X = 4, continue loop
62          bne MoveSpritesUp
63  CheckUpButtonDone:
64          rep #$20                        ; set A to 16-bit
65
66  CheckDownButton:
```

```
67          lda #$0000                      ; set A to zero
68          ora (Trigger)                   ; check whether the down button wa
69          ora (Held)                      ; ...or held from last frame
70          and #DOWN_BUTTON
71          beq CheckDownButtonDone         ; if neither has occured, move on
72          ; else, move sprites down
73          ldx #$0000                      ; X is the loop counter
74          ldy #$0001                      ; Y is the offset into the OAM mir
75          sep #$20                        ; set A to 8-bit
76          ; check if sprites move below buttom boundry
77          lda OAMMIRROR, Y
78          clc
79          adc VER_SPEED
80          cmp #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
81          bcs CorrectVerticalPositionUp
82  MoveSpritesDown:
83          lda OAMMIRROR, Y
84          clc
85          adc VER_SPEED
86          sta OAMMIRROR, Y
87          iny                             ; increment Y by 4
88          iny
89          iny
90          iny
91          inx
92          cpx #$0004                      ; unless X = 4, continue loop
93          bne MoveSpritesDown
94  CheckDownButtonDone:
95          rep #$20                        ; set A to 16-bit
96          jmp CheckLeftButton             ; continue input check
97
98  CorrectVerticalPositionDown:
99          sep #$20                        ; set A to 8-bit
100         lda #$0000                      ; set A to zero
101         stz OAMMIRROR + 1               ; sprite 1, vertical position
102         stz OAMMIRROR + 5               ; sprite 3, vertical position
103         lda #SPRITE_SIZE
104         sta OAMMIRROR + 9               ; sprite 2, vertical position
105         sta OAMMIRROR + 13              ; sprite 4, vertical position
106         rep #$20                        ; set A to 16-bit
107         jmp CheckLeftButton             ; continue input check
108
109 CorrectVerticalPositionUp:
110         sep #$20                        ; set A to 8-bit
```

```
111         lda #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
112         sta OAMMIRROR + 1                   ; sprite 1, vertical position
113         sta OAMMIRROR + 5                   ; sprite 3, vertical position
114         lda #(SCREEN_BOTTOM - SPRITE_SIZE)
115         sta OAMMIRROR + 9                   ; sprite 2, vertical position
116         sta OAMMIRROR + 13                  ; sprite 4, vertical position
117         rep #$20                            ; set A to 16-bit
118
119 CheckLeftButton:
120         lda #$0000                          ; set A to zero
121         ora (Trigger)                       ; check whether the up button was
122         ora (Held)                          ; ...or held from last frame
123         and #LEFT_BUTTON
124         beq CheckLeftButtonDone             ; if neither has occured, move on
125         ; else, move sprites up
126         ldx #$0000                          ; X is the loop counter
127         ldy #$0000                          ; Y is the offset into the OAM mir
128         sep #$20                            ; set A to 8-bit
129 MoveSpritesLeft:
130         lda OAMMIRROR, Y
131         sec
132         sbc HOR_SPEED
133         bcc CorrectHorizontalPositionRight
134         sta OAMMIRROR, Y
135         iny                                 ; increment X by 4
136         iny
137         iny
138         iny
139         inx
140         cpx #$0004                          ; unless Y = 4, continue loop
141         bne MoveSpritesLeft
142 CheckLeftButtonDone:
143         rep #$20                            ; set A to 16-bit
144
145 CheckRightButton:
146         lda #$0000                          ; set A to zero
147         ora (Trigger)                       ; check whether the down button wa
148         ora (Held)                          ; ...or held from last frame
149         and #RIGHT_BUTTON
150         beq CheckRightButtonDone            ; if neither has occured, move on
151         ; else, move sprites down
152         ldx #$0000                          ; X is the loop counter
153         ldy #$0000                          ; Y is the offset into the OAM mir
154         sep #$20                            ; set A to 8-bit
```

```
155         ; check whether sprites move beyond right boundry
156         lda OAMMIRROR, Y
157         clc
158         adc HOR_SPEED
159         cmp #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
160         bcs CorrectHorizontalPositionLeft
161 MoveSpritesRight:
162         lda OAMMIRROR, Y
163         clc
164         adc HOR_SPEED
165         sta OAMMIRROR, Y
166         iny                             ; increment X by 4
167         iny
168         iny
169         iny
170         inx
171         cpx #$0004                      ; unless Y = 4, continue loop
172         bne MoveSpritesRight
173 CheckRightButtonDone:
174         rep #$20                        ; set A to 16-bit
175         jmp InputDone
176
177 CorrectHorizontalPositionRight:
178         sep #$20                        ; set A to 8-bit
179         stz OAMMIRROR + 0               ; sprite 1, horizontal position
180         stz OAMMIRROR + 8               ; sprite 2, horizontal position
181         lda #SPRITE_SIZE
182         sta OAMMIRROR + 4               ; sprite 3, horizontal position
183         sta OAMMIRROR + 12              ; sprite 4, horizontal position
184         jmp InputDone
185
186 CorrectHorizontalPositionLeft:
187         sep #$20                        ; set A to 8-bit
188         lda #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
189         sta OAMMIRROR + 0               ; sprite 1, horizontal position
190         sta OAMMIRROR + 8               ; sprite 2, horizontal position
191         lda #(SCREEN_RIGHT - SPRITE_SIZE)
192         sta OAMMIRROR + 4               ; sprite 3, horizontal position
193         sta OAMMIRROR + 12              ; sprite 4, horizontal position
194
195 InputDone:
196         sep #$20                        ; set A back to 8-bit
197         pld                             ; restore D...
198         plx                             ; ...and X registers
```

```
199
200         rts
201 .endproc
202 ;----------------------------------------------------------------------------
```

Input.s hosted with ❤ by **GitHub** **view raw**

Then we replace the main game loop `GameLoop` with this:

```
1  ; ----------------------------------------------------------------------------
2  ;   File: Main.s
3  ;   Description: Displays a sprite movable with the dpad
4  ; ----------------------------------------------------------------------------
5
6  ;----- Export ----------------------------------------------------------------
7  .export     ResetHandler        ; export the entry point of the demo
8  .export     NMIHandler
9  ;----------------------------------------------------------------------------
10
11 ;----- Assembler Directives --------------------------------------------------
12 .p816                          ; tell the assembler this is 65816 code
13 ;----------------------------------------------------------------------------
14
15 ;----- Includes --------------------------------------------------------------
16 .include "Assets.inc"
17 .include "GameConstants.inc"
18 .include "Init.inc"
19 .include "InitSNES.inc"
20 .include "Input.inc"
21 .include "Joypad.inc"
22 .include "MemoryMapWRAM.inc"
23 .include "PPU.inc"
24 .include "Registers.inc"
25 ;----------------------------------------------------------------------------
26
27 .segment "CODE"
28 ;----------------------------------------------------------------------------
29 ;   This is the entry point of the demo
30 ;----------------------------------------------------------------------------
31 .proc   ResetHandler
32         sei                     ; disable interrupts
33         clc                     ; clear the carry flag
34         xce                     ; switch the 65816 to native (16-bit mode)
35         rep #$10                ; set X and Y to 16-bit
```

```
36          sep #$20                ; set A to 8-bit
37          lda #$8f                ; force v-blanking
38          sta INIDISP
39          stz NMITIMEN            ; disable NMI
40          ; set the stack pointer to $1fff
41          ldx #$1fff              ; load X with $1fff
42          txs                     ; copy X to stack pointer
43
44          jsr InitDemo            ; initialize the demo
45          jmp GameLoop            ; enter main game loop
46  .endproc
47  ;--------------------------------------------------------------------------
48
49  ;--------------------------------------------------------------------------
50  ;   Executed during V-Blank
51  ;--------------------------------------------------------------------------
52  .proc   GameLoop
53          wai                                 ; wait for NMI / V-Blank
54
55          ; read joypad 1
56          ; push addresses of joy pad data to stack
57          ldx #JOYHELD1
58          phx
59          ldx #JOYTRIGGER1
60          phx
61          ldx #JOYRAW1
62          phx
63          jsr ReadJoypad1
64          ; stack clean up
65          plx
66          plx
67          plx
68
69          ; push addresses of joypad 1 data to stack
70          ldx #JOYHELD1
71          phx
72          ldx #JOYTRIGGER1
73          phx
74          ldx #JOYRAW1
75          phx
76          jsr HandleInput
77          ; stack clean up
78          plx
79          plx
```

```
80          plx
81
82          jmp GameLoop
83  .endproc
84  ;---------------------------------------------------------------------------
85
86  ;---------------------------------------------------------------------------
87  ;   Will be called during V-Blank every frame
88  ;---------------------------------------------------------------------------
89  .proc   NMIHandler
90          lda RDNMI               ; read NMI status, acknowledge NMI
91
92          tsx                     ; save old stack pointer
93          pea OAMMIRROR           ; push mirror address to stack
94          jsr UpdateOAMRAM        ; update OAMRAM
95          txs                     ; restore old stack pointer
96
97          rti
98  .endproc
99  ;---------------------------------------------------------------------------
100
101 ;---------------------------------------------------------------------------
102 ;   Is not used in this program
103 ;---------------------------------------------------------------------------
104 .proc   IRQHandler
105         ; code
106         rti
107 .endproc
```

Main.s hosted with ❤ by **GitHub** **view raw**

Finally, rename `src/JoypadSprites.s` to `src/Main.s`, and make sure you update your `CMakeLists.txt` files accordingly. Build and test your project to ensure all is still working as previously. If you get lost or confused, check out the SNES Assembly Adventure repository on GitHub.

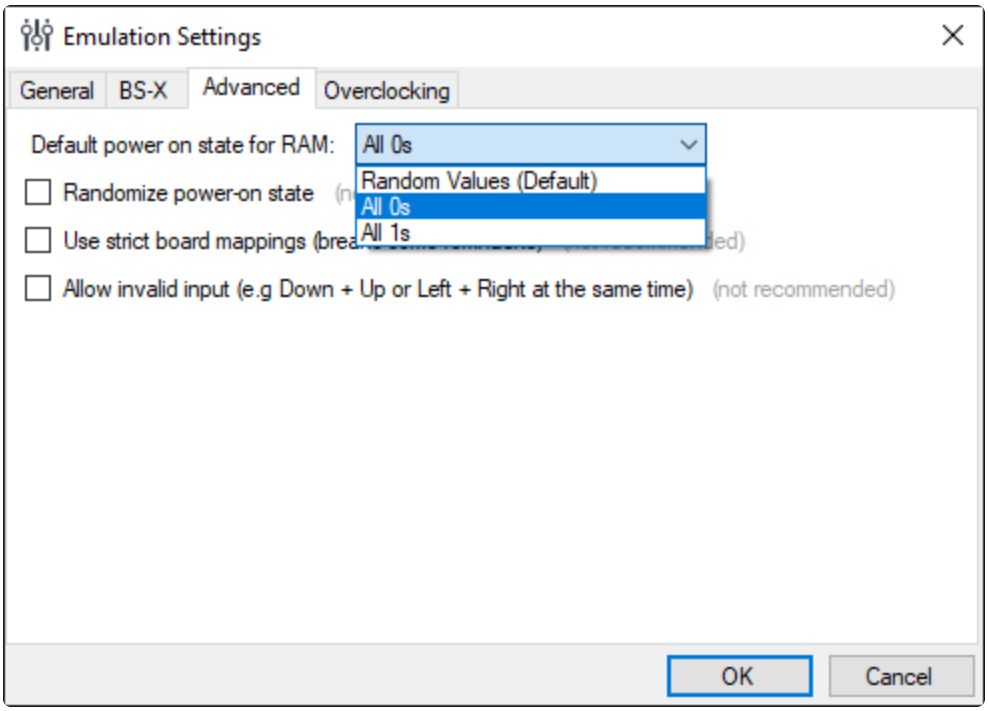
Updating `Registers.inc`

This is an as good time as any to add the missing registers to `src/common/Registers.inc`. It follows the naming convention found in the SNES Development Manual. There are also a number of named constants at the end of the file that will help in the future. They're meant to be used as bitmasked when working with registers. I'm not reproducing the entire file here, it's

just a boring list of names and numbers. Check the SNES Assembly Adventure repository on GitHub for the full file.

Initializing the SNES to a Defined State

So far, we've taken advantage of the emulator. We told it to set all memory to `0`s all the way back in part 1:



Mesen-S(X) Emulation Settings

While this is rather convenient when working with an emulator that supports that, it, unfortunately, means that our code couldn't run on actual hardware. If we set the "Default power on state RAM" option back to "Random Values (Default)", we'll find that every time we load our current demo code, we'll see a random background color. That's obviously not what we want.

The solution is to set all registers to a standard/initial value and clear all memory (VRAM, CGRAM, and OAMRAM) to a known value. To do this, we introduce four new helper subroutines, defined in files `src/common/InitSNES.{inc,s}`:

```
1  .ifndef INIT_SNES_INC
2  INIT_SNES_INC = 1
3  ; ------------------------------------------------------------------------------
4  ;   File: InitSNES.inc
5  ;   Description: Reset the SNES to a known state
```

```
; ------------------------------------------------------------------------

;----- Import ------------------------------------------------------------
.import     ClearRegisters      ; Clear all PPU and CPU registers of the SNES
.import     ClearVRAM           ; Clear the complete VRAM to $00
.import     ClearCGRAM          ; Clear CG-RAM to $00 (black)
.import     ClearOAMRAM         ; Clear OAM-RAM to $ff (all sprites off screen)
;------------------------------------------------------------------------
```

InitSNES.inc hosted with ❤ by **GitHub** **view raw**

```
; ------------------------------------------------------------------------
;   File: InitSNES.s
;   Description: Reset the SNES to a known state
; ------------------------------------------------------------------------

;----- Export ------------------------------------------------------------
.export     ClearRegisters      ; Clear all PPU and CPU registers of the SNES
.export     ClearVRAM           ; Clear the complete VRAM to $00
.export     ClearCGRAM          ; Clear CG-RAM to $00 (black)
.export     ClearOAMRAM         ; Clear OAM-RAM to $ff (all sprites off screen
;------------------------------------------------------------------------

;----- Assembler Directives ----------------------------------------------
.p816                           ; tell the assembler this is 65816 code
.a8
.i16
;------------------------------------------------------------------------

;----- Includes ----------------------------------------------------------
.include "Registers.inc"
;------------------------------------------------------------------------

.segment "CODE"
;------------------------------------------------------------------------
;   Clear all PPU and CPU registers to standard values
;------------------------------------------------------------------------
.proc   ClearRegisters
        ; PPU registers
        ldx #$2101                  ; registers $2101 through $210c
loop1:  stz $00, X
        inx
        cpx #$210d
```

```
33          bne loop1
34
35  loop2:  stz $00, X                      ; registers $210d through $2114, two w
36          stz $00, X
37          inx
38          cpx #$2115
39          bne loop2
40
41          lda #$80                        ; VRAM address increment to write to V
42          sta VMAINC
43
44          inx                             ; registers $2116 through 211a
45  loop3:  stz $00, X
46          inx
47          cpx #$211b
48          bne loop3
49
50          lda #$01                        ; registers $211b through $2120
51  loop4:  sta $00, X
52          stz $00, X
53          inx
54          stz $00, X
55          stz $00, X
56          inx
57          stz $00, X
58          stz $00, X
59          inx
60          cpx #$2121
61          bne loop4
62
63  loop5:  stz $00, X                      ; registers $2121 through $2133
64          inx
65          cpx #$2134
66          bne loop5
67
68          ; special values
69          lda #$30
70          sta CGSWSEL
71          lda #$e0
72          sta COLDATA
73
74          ; CPU registers
75          stz NMITIMEN
76          lda #$ff
```

```
77          sta WRIO
78
79          ldx #$4202                  ; registers $4202 through $420d
80  loop6:  stz $00, X
81          inx
82          cpx #$420e
83          bne loop6
84
85          rts
86  .endproc
87  ;----- end of subroutine ClearRegisters -------------------------------------
88
89  ;------------------------------------------------------------------------------
90  ;   Clear the complete VRAM to $00
91  ;------------------------------------------------------------------------------
92  .proc   ClearVRAM
93          ; set up VRAM registers
94          lda #$80                    ; increment after write to VMDATAH by
95          sta VMAINC
96          ldx #$0000                  ; set VRAM address to $0000
97          stx VMADDL
98          ; set up DMA channel 0 for transfer
99          lda #$09                    ; 2-byte write and fixed address
100         sta DMAP0
101         lda #$18                    ; set destination to $(21)18
102         sta BBAD0
103         stz $0000                   ; store $00 in WRAM $0000
104         stz A1T0L                   ; set source address to $00:0000
105         stz A1T0H
106         stz A1T0B
107         stz DAS0L                   ; set number of bytes to transfer to $
108         stz DAS0H
109         lda #$01
110         sta DAS0B
111         sta MDMAEN                  ; start transfer
112
113         rts
114 .endproc
115 ;----- end of subroutine ClearVRAM ------------------------------------------
116
117 ;------------------------------------------------------------------------------
118 ;   Clear the complete CG-RAM to $00
119 ;------------------------------------------------------------------------------
120 .proc   ClearCGRAM
```

```
121         ; set up CG-RAM registers
122         stz CGADD                       ; set CG-RAM address to $00
123         ; write 512/$120 byte of $00 to CG-RAM by DMA channel 0
124         lda #$0a                        ; 2-byte write to fixed address and fi
125         sta DMAP0
126         lda #$22                        ; set destination to CGDATA
127         sta BBAD0
128         stz $0000                       ; set WRAM $00:0000 to $00
129         stz A1T0L                       ; set source address to $00:0000
130         stz A1T0H
131         stz A1T0B
132         stz DAS0L                       ; set number of bytes to transfer to 5
133         lda #$02
134         sta DAS0H
135         stz DAS0B
136         lda #$01                        ; start transfer
137         sta MDMAEN
138
139         rts
140 .endproc
141 ;----- end of subroutine ClearCGRAM ----------------------------------------
142
143 ;-------------------------------------------------------------------------
144 ;   Clear the complete OAM-RAM to $ff, which moves all sprites off screen
145 ;-------------------------------------------------------------------------
146 .proc   ClearOAMRAM
147         ; set up OAM-RAM registers
148         stz OAMADDL                     ; set OAM-RAM address to $0000
149         stz OAMADDH
150         ; write 544/$220 words to OAM-RAM by DMA channel 0
151         lda #$0a                        ; 2-byte write to fixed address and fi
152         sta DMAP0
153         lda #$04                        ; set destination address to OAMDATA
154         sta BBAD0
155         lda #$ff                        ; set WRAM $00:0000 to $ff
156         sta $0000
157         stz A1T0L                       ; set destination address to $00:0000
158         stz A1T0H
159         stz A1T0B
160         lda #$20                        ; set number of bytes to transfer to 5
161         sta DAS0L
162         lda #$02
163         sta DAS0H
164         stz DAS0B
```

```
165         lda #$01                        ; start transfer
166         sta MDMAEN
167
168         rts
169 .endproc
170 ;----- end of subroutine ClearOAMRAM ---------------------------------------
```

InitSNES.s hosted with ❤ by **GitHub** **view raw**

These four subroutines are pretty straightforward. You should have no trouble understanding what is going on here.

`ClearRegisters` (line 27) sets all registers to the value recommended in chapter 26 of the SNES Development Manual. It uses a handful of loops to traverse some address ranges more concisely.

`ClearVRAM` (line 92) and `ClearCGRAM` (line 120) are almost identical. The idea is to set up a DMA (Direct Memory Access, see part 7 if you need a refresher) and transfer a piece of constant data (here, simply zero/ `$00`) to fill out the video and color RAM, respectively.

`ClearOAMRAM` (line 146) also uses DMA to transfer data to object attribute memory. But we need to take some extra care here. Remember that the SNES has a limit of 32 sprites per scanline. If we were to set all of OAM RAM to zero, that would result in all 128 possible sprites being rendered in the top left corner (that is, position `(0, 0)`). This could lead to some ugly rendering errors. The solution here is to move all sprites off-screen. So instead of clearing OAM RAM to `$00`, we'll clear it to `$ff` - this will result in all sprites starting at position (-128, 241), that is, outside of the visible range. It is then for the actual game code to set these values correctly. Here, we just don't want any sprites on screen when the console launches.

Final Touches

Finally, add the four new subroutines to the `ResetHandler` in `src/Main.s`:

```
1  ; ----------------------------------------------------------------------------
2  ;   File: Main.s
3  ;   Description: Displays a sprite movable with the dpad
4  ; ----------------------------------------------------------------------------
5
6  ;----- Export ----------------------------------------------------------------
7  .export     ResetHandler        ; export the entry point of the demo
8  .export     NMIHandler
9  ;----------------------------------------------------------------------------
```

```
10
11  ;----- Assembler Directives ---------------------------------------------------
12  .p816                           ; tell the assembler this is 65816 code
13  ;------------------------------------------------------------------------------
14
15  ;----- Includes ---------------------------------------------------------------
16  .include "Assets.inc"
17  .include "GameConstants.inc"
18  .include "Init.inc"
19  .include "InitSNES.inc"
20  .include "Input.inc"
21  .include "Joypad.inc"
22  .include "MemoryMapWRAM.inc"
23  .include "PPU.inc"
24  .include "Registers.inc"
25  ;------------------------------------------------------------------------------
26
27  .segment "CODE"
28  ;------------------------------------------------------------------------------
29  ;   This is the entry point of the demo
30  ;------------------------------------------------------------------------------
31  .proc   ResetHandler
32          sei                     ; disable interrupts
33          clc                     ; clear the carry flag
34          xce                     ; switch the 65816 to native (16-bit mode)
35          rep #$10                ; set X and Y to 16-bit
36          sep #$20                ; set A to 8-bit
37          lda #$8f                ; force v-blanking
38          sta INIDISP
39          stz NMITIMEN            ; disable NMI
40          ; set the stack pointer to $1fff
41          ldx #$1fff              ; load X with $1fff
42          txs                     ; copy X to stack pointer
43
44          ; initialize SNES
45          jsr ClearRegisters
46          jsr ClearVRAM
47          jsr ClearCGRAM
48          jsr ClearOAMRAM
49
50          jsr InitDemo            ; initialize the demo
51          jmp GameLoop            ; enter main game loop
52  .endproc
53  ;------------------------------------------------------------------------------
```

```
54
55  ;-------------------------------------------------------------------------
56  ;   Executed during V-Blank
57  ;-------------------------------------------------------------------------
58  .proc   GameLoop
59          wai                                 ; wait for NMI / V-Blank
60
61          ; read joypad 1
62          ; push addresses of joy pad data to stack
63          ldx #JOYHELD1
64          phx
65          ldx #JOYTRIGGER1
66          phx
67          ldx #JOYRAW1
68          phx
69          jsr ReadJoypad1
70          ; stack clean up
71          plx
72          plx
73          plx
74
75          ; push addresses of joypad 1 data to stack
76          ldx #JOYHELD1
77          phx
78          ldx #JOYTRIGGER1
79          phx
80          ldx #JOYRAW1
81          phx
82          jsr HandleInput
83          ; stack clean up
84          plx
85          plx
86          plx
87
88          jmp GameLoop
89  .endproc
90  ;-------------------------------------------------------------------------
91
92  ;-------------------------------------------------------------------------
93  ;   Will be called during V-Blank every frame
94  ;-------------------------------------------------------------------------
95  .proc   NMIHandler
96          lda RDNMI               ; read NMI status, acknowledge NMI
97
```

```
98          tsx                     ; save old stack pointer
99          pea OAMMIRROR           ; push mirror address to stack
100         jsr UpdateOAMRAM        ; update OAMRAM
101         txs                     ; restore old stack pointer
102
103         rti
104 .endproc
105 ;---------------------------------------------------------------------------
106
107 ;---------------------------------------------------------------------------
108 ;   Is not used in this program
109 ;---------------------------------------------------------------------------
110 .proc   IRQHandler
111         ; code
112         rti
113 .endproc
```

Main.s hosted with ❤ by **GitHub** **view raw**

Make sure to update all `CMakeLists.txt` files accordingly. Your final file structure should look like this:

```
snes_cradle/
    assets/
        Assets.inc
        Assets.s
        CMakeLists.txt
        SpriteColors.pal
        Sprites.vra
    cmake/
        CMakeCA65816Compiler.cmake.in
        CMakeCA65816Information.cmake
        CMakeDetermineCA65816Compiler.cmake
        CMakeTestCA65816Compiler.cmake
    src/
        common/
            CMakeLists.txt
            GameConstants.inc
            InitSNES.inc
            InitSNES.s
```

```
            Joypad.inc
            Joypad.s
            MemoryMapWRAM.inc
            Registers.inc
            Vector.s
        Game/
            CMakeLists.txt
            Init.inc
            Init.s
            Input.inc
            Input.s
        PPU/
            CMakeLists.txt
            PPU.inc
            PPU.s
        CMakeLists.txt
        Main.s
    CMakeLists.txt
    MemoryMap.cfg
```

Check the code on GitHub if you're in doubt.

Conclusion

We did a lot of housekeeping and refactoring. Nothing really new happened here, but this little template should allow us to skip the initial setup each new project needs. So we can focus on writing our next demo or game. Which is what we will do next.

Next time, we'll be learning a bit more about sprites, sprite ordering/priority, and how to generate color palettes on the fly.

Links and References

- undisbeliever has a great overview of all 65816 opcodes, you should bookmark this
- The official SNES Development Manual

ALSO ON MACHINE CODE CONSTRUCTION YARD

SNES Assembly Adventure 08: …

4 years ago · 1 comment

I do assembly programming on old machines and boat building.

A CMake Template for Game Boy Development

3 years ago · 1 comment

I do assembly programming on old machines and boat building.

SNES Assembly Adventure 05: …

8 years ago · 8 comments

I do assembly programming on old machines and boat building.

SNES Assem Adventure 0

8 years ago · 16

I do assembly on old machin building.